Week 15 Technical Reflection

Laurence Matthew B Manuel BSIT 3A PFEBSIT 002 Project 29 Week 15

Decision with the greatest effect

The decision with the greatest effect was to make timeout advice depend on whether the request had an idempotency key. The original Flutter client told the user to retry the same form after every timeout. The new test showed why that was unsafe for operations such as secure receive or action changes: the client might not know whether the server completed the first request. The corrected client advises a same-key retry only for the idempotent consultation request and tells the user to inspect current state before retrying other operations. The two targeted timeout tests and duplicate API tests confirmed this distinction.

Most difficult failure or limitation

The most useful unexpected failure was the login layout regression. I added visible email and password counters to match server limits, but the counters pushed the sign-in button below a 600-pixel test viewport. That single layout change caused several later widget tests to fail because they could not sign in. I improved the design by keeping explicit validators without the extra counters, then reran every Flutter test. A future test improvement would make hit-test warnings fatal and include an explicit 600-pixel-height login check so the root cause appears immediately instead of through downstream failures.

Work I can explain and modify

I can explain and modify the Laravel pagination and early authorization tests in `backend/tests/Feature/Week15HardeningTest.php`. The test file creates controlled roles, verifies page bounds and metadata, proves a disallowed receiver gets 403 before cryptographic detail, and checks response hardening headers. The related controller and middleware changes are small enough to trace from request through validation, authorization, response, and evidence.

What a maintainer should verify next

An authorized maintainer should verify concurrent writes, action-item idempotency, per-user mutation limits, and reverse-proxy header behavior in the Week 16 nonproduction environment. This work is outside the current evidence because safe thresholds require a documented workload and deployed monitoring, while the current Week 15 environment is one local workstation. The maintainer should also test the target browser and assistive-technology combinations before broader use.

Closing reflection

Week 15 changed my view of testing from proving that the normal path works to building a chain from requirement, expected result, observed failure, fix, and regression. Keeping both the initial failures and the layout regression made the final passing counts more credible. AI assistance was used and reviewed against the module, source, and executed evidence; I remain responsible for understanding and explaining the submitted artifacts.